

Cocktail Assistant

Sistema de Recuperación de Información con Arquitectura
Multiagente
basado en Retrieve-Augmented Generation

Amalia Beatriz Valiente Hinojosa C312
Jorge Alejandro Echevarría Brunet C312
Rodrigo Mederos González C311
Noel Pérez Calvo C311

Universidad de La Habana, Facultad de Matemática y Computación

18 de junio de 2025

Resumen Este documento presenta el desarrollo de un sistema de recuperación de información especializado en el dominio de bartender, implementado mediante una arquitectura multiagente alineada con el modelo Retrieve-Augmented Generation (RAG). El sistema integra diversos componentes que incluyen: procesamiento de lenguaje natural para la interpretación de consultas y generación de respuestas, técnicas meta-heurísticas para optimizar la recuperación de información, representación del conocimiento mediante una ontología especializada, y un crawler automatizado para la recopilación y actualización dinámica de datos. El informe detalla el diseño e implementación del sistema, justificando cada decisión técnica tomada, y presenta una evaluación cuantitativa y cualitativa de los resultados obtenidos, así como un análisis crítico de las fortalezas y limitaciones del trabajo realizado.

1. Introducción

Se desarrolló un sistema de recuperación de información especializado en el dominio de bartender, implementando una arquitectura multiagente que permite la coordinación y colaboración entre componentes especializados. El sistema se alinea con la arquitectura genérica del modelo Retrieve-Augmented Generation (RAG), combinando capacidades avanzadas de recuperación de información con generación de texto.

El sistema aborda el desafío de proporcionar información precisa, actualizada y contextualmente relevante sobre cócteles, ingredientes, técnicas de preparación

y otros aspectos relacionados con la coctelería. Mediante la integración de diversas tecnologías, se busca ofrecer respuestas coherentes y fundamentadas, capaces de asistir tanto a profesionales como a entusiastas de la coctelería.

El código fuente completo del proyecto se encuentra disponible en el repositorio GitHub: <https://github.com/noelpc03/CocktailAssistant>.

Como referencia principal para la arquitectura RAG se utilizó [2] y [3]. Por ejemplo, el trabajo de Lewis et al. [3] es fundamental en este campo. Para los sistemas multiagente, nos basamos en los principios establecidos por Wooldridge [5].

2. Diseño e Implementación

El sistema **Cocktail Assistant** se diseñó siguiendo una arquitectura multi-agente [5], donde cada agente cumple una función especializada y se comunica con los demás para resolver consultas complejas en el dominio de la coctelería. Esta aproximación permite desacoplar responsabilidades, facilitar la escalabilidad y optimizar el procesamiento de información tanto estructurada como no estructurada [4].

2.1. Sistema Multiagente

El sistema Cocktail Assistant está construido sobre una arquitectura multi-agente donde cada agente implementa la interfaz base **Agent** definida en `agents.common.agent_interface`, proporcionando métodos estandarizados para `start()`, `stop()` y `process_message()`. Esta estructura permite una fácil ampliación del sistema y una comunicación uniforme. A continuación, se describen los agentes principales con detalles de su implementación técnica:

Agente Coordinador El *agente coordinador* (implementado en `coordinator_agent.py`) actúa como núcleo central del sistema, orquestando la interacción entre todos los agentes. Opera mediante un sistema de enrutamiento dinámico para la gestión de mensajes, distribuyendo consultas a agentes específicos según el tipo de acción solicitada y utilizando el patrón de diseño broker a través de la clase **MessageBroker**. Para el seguimiento eficiente de operaciones, mantiene un diccionario de operaciones activas que permite el control asíncrono de tareas y gestiona sus correspondientes callbacks de finalización. El agente integra además un sistema de verificación mediante el módulo **verify_and_enhance_response**, que evalúa la calidad y precisión de las respuestas generadas. Una de sus funciones críticas es la gestión de resultados ontológicos a través del módulo **handle_ontology_results**, transformando consultas SPARQL complejas en respuestas estructuradas y comprensibles para el usuario final.

Agente Crawler El *agente crawler* (implementado en `crawler_agent.py`) constituye un componente fundamental para la adquisición de datos, utilizando el algoritmo de Optimización de Colonia de Hormigas (ACO) [1] para explorar eficientemente la web. Su núcleo es la clase `AntColonyCrawler`, que gestiona múltiples "hormigas" virtuales para explorar el espacio web en paralelo, con parámetros configurables para la importancia de feromonas (`alpha`) e importancia heurística (`beta`). El sistema evalúa candidatos de exploración mediante un mecanismo de ponderación que considera la presencia de palabras clave de coctelería, la calidad del dominio según fuentes confiables predefinidas, y el contenido del texto ancla. La exploración se gestiona mediante una matriz de feromonas que asocia cada URL con un nivel de importancia, actualizado dinámicamente según la calidad de información encontrada y sujeto a una tasa de evaporación para favorecer la exploración continua. El agente implementa además un sistema de filtrado mediante expresiones regulares, eliminando contenido irrelevante y enfocándose en texto significativo.

Agente Crawler Dinámico El *agente crawler dinámico*, implementado en el archivo `dynamic_crawler.py`, complementa al crawler principal añadiendo capacidades de búsqueda en tiempo real cuando se detectan lagunas de conocimiento en el sistema. Su funcionamiento se basa en un sofisticado sistema de verificación de respuestas que utiliza un modelo de lenguaje para evaluar la calidad y completitud de las respuestas generadas, produciendo una valoración estructurada que determina si se necesita información adicional. Cuando identifica información inadecuada o incompleta, emplea un mecanismo de búsqueda contextual que genera consultas refinadas utilizando la respuesta original y la pregunta del usuario para buscar específicamente la información faltante. Para obtener datos actualizados, implementa la integración con APIs externas, comunicándose con servicios web para obtener resultados de búsqueda, realizando parsing y formateo de nuevas fuentes de información, utilizando una clave API almacenada en un archivo independiente. El agente cuenta además con un sistema de procesamiento adaptativo que implementa reintentos y reprocesamiento de respuestas cuando determina que la información original no es satisfactoria, garantizando así la calidad y relevancia de los resultados finales.

Agente de Generación El *agente de generación*, implementado en el archivo `generation_agent.py`, utiliza modelos de lenguaje de última generación para producir respuestas coherentes y precisas. Este componente integra la tecnología de Mistral AI, configurando y utilizando modelos específicos (`tiny`, `small`, `medium`) con parámetros cuidadosamente ajustados como temperatura (0.6) y top-p (0.95) para lograr un equilibrio óptimo entre creatividad y precisión factual. El agente implementa un sistema avanzado de formateo de contexto que estructura la información recuperada de manera eficiente, optimizando el contexto proporcionado al modelo y maximizando así la relevancia y coherencia de las respuestas generadas. Una característica destacable es su capacidad de procesamiento multilingüe, que incluye detección automática del idioma de la consul-

ta y funcionalidades de traducción cuando es necesario, permitiendo atender a usuarios con diferentes preferencias idiomáticas. Además, incorpora mecanismos robustos de manejo de errores que proporcionan degradación elegante cuando el modelo principal no está disponible, ofreciendo respuestas alternativas mediante templates predefinidos, garantizando así la disponibilidad continua del servicio incluso en condiciones adversas.

Agente de Ontologías El *agente de ontologías*, implementado en el archivo `ontology_agent.py`, es responsable de la representación semántica del conocimiento en el dominio de la coctelería. Este componente adopta un enfoque basado en estándares semánticos, utilizando la biblioteca `rdflib` para construir, manipular y consultar grafos RDF, con espacios de nombres personalizados específicamente definidos para el dominio de coctelería. Una de sus funcionalidades principales es la extracción de tripletas, implementando procesos sofisticados para extraer relaciones semánticas de texto mediante el módulo `extract_sparql`, lo que permite transformar conocimiento no estructurado en tripletas RDF formalizadas como sujeto-predicado-objeto. El agente proporciona además un potente motor de consultas SPARQL que ofrece una interfaz para traducir preguntas formuladas en lenguaje natural a consultas SPARQL ejecutables, permitiendo así realizar consultas complejas y estructuradas sobre la ontología. Para facilitar la interpretación de resultados, incluye capacidades de visualización a través del módulo `generate_visualization`, que crea representaciones gráficas de la ontología completa o de subconjuntos relevantes para una consulta específica, mejorando la comprensión de las relaciones semánticas identificadas.

Agente de Recuperación El *agente de recuperación*, implementado en el archivo `retrieval_agent.py`, gestiona la búsqueda eficiente en el espacio vectorial, constituyendo un componente crítico para la recuperación de información relevante [4]. Este agente implementa un sistema de indexación avanzado utilizando la biblioteca FAISS (Facebook AI Similarity Search), lo que le permite crear índices de alta eficiencia para realizar búsquedas rápidas por similitud en espacios vectoriales de alta dimensionalidad. Para facilitar la presentación de resultados significativos al usuario, mantiene un sofisticado sistema de mapeo entre los vectores de embeddings y los metadatos de documentos, incluyendo URLs, títulos y fechas. La precisión del sistema se optimiza mediante un umbral adaptativo de similitud (configurado en 0.2), que filtra automáticamente resultados poco relevantes para las consultas realizadas. Una característica importante para garantizar la eficiencia operativa es su sistema de persistencia, que incluye funcionalidades para guardar y cargar índices vectoriales, permitiendo así la continuidad del servicio sin necesidad de reindexar todos los documentos cada vez que se reinicia el sistema.

Agente de Búsqueda El *agente de búsqueda*, implementado en el archivo `search_agent.py`, actúa como interfaz entre las consultas de usuario y el sistema de recuperación, facilitando la interacción con el repositorio de conocimiento.

Este componente implementa un sofisticado sistema de procesamiento de consultas que normaliza y prepara las entradas del usuario para su transformación en embeddings vectoriales compatibles con el sistema de recuperación subyacente. La gestión de resultados constituye otra funcionalidad clave, donde el agente ordena, filtra y formatea los resultados de búsqueda para presentar la información más relevante en primer lugar, incorporando una capacidad configurable para mostrar puntuaciones de confianza que ayudan al usuario a evaluar la calidad de las respuestas. Una característica particularmente valiosa es su capacidad de integración múltiple, que le permite coordinar resultados provenientes de distintas fuentes (como el sistema vectorial y la ontología) cuando es necesario, consolidando la información heterogénea en un formato coherente y comprensible para el usuario final.

Agente de Estrategia El *agente de estrategia*, implementado en el archivo `strategy_agent.py`, constituye un componente clave que determina la ruta óptima para procesar cada consulta recibida en el sistema. Este agente implementa un sistema avanzado de análisis de consultas que utiliza un modelo de lenguaje configurado con baja temperatura (0.1), lo que garantiza decisiones consistentes y predecibles sobre la naturaleza y el tipo de cada consulta procesada. Para optimizar el rendimiento computacional, especialmente en escenarios de uso repetitivo, incorpora un eficiente sistema de caché implementado mediante un diccionario que almacena decisiones previas para consultas similares, reduciendo significativamente el tiempo de respuesta. El componente central de su funcionamiento es un mecanismo de selección multicriterio que evalúa diversos factores como la especificidad de la consulta, la presencia de entidades nombradas específicas del dominio, y la estructura lingüística de la pregunta, para determinar la estrategia de recuperación óptima en cada caso particular.

Agente Vectorizador El *agente vectorizador*, implementado en el archivo `vectorizer_agent.py`, cumple la función fundamental de transformar texto en representaciones numéricas vectoriales que permiten el procesamiento semántico de la información. Este componente integra un modelo Transformer de última generación, específicamente `all-mpnet-base-v2` de Sentence Transformers, cargado a través de la infraestructura de Hugging Face para generar embeddings de alta calidad semántica. Para manejar documentos extensos, implementa un sofisticado sistema de chunking adaptativo que divide los textos en fragmentos de tamaño configurable (512 tokens) con una superposición controlada (50 tokens), lo que permite mantener la coherencia contextual entre fragmentos adyacentes. La eficiencia computacional se optimiza mediante un sistema de procesamiento por lotes que vectoriza múltiples documentos en paralelo, aprovechando operaciones tensoriales eficientes que maximizan el uso de recursos de hardware. Además, incorpora un mecanismo de carga lazy que implementa la inicialización bajo demanda del modelo, cargándolo solo cuando es estrictamente necesario durante las operaciones de vectorización, lo que reduce significativamente el consumo de memoria en períodos de inactividad.

La arquitectura multiagente descrita proporciona modularidad, especialización y robustez al sistema, donde cada agente puede evolucionar independientemente mientras mantiene interfaces estandarizadas para la comunicación y cooperación. La implementación concreta de cada agente aprovecha bibliotecas especializadas y algoritmos avanzados para sus respectivas áreas de responsabilidad, permitiendo una experiencia cohesiva y eficiente para el usuario final. Esta división de responsabilidades facilita además la escalabilidad del sistema y la integración de nuevas funcionalidades sin afectar la estabilidad de los componentes existentes.

2.2. Implementación del Crawler y Gestión de Datos

El componente de recuperación de datos del sistema se implementó a través de dos agentes complementarios: el Crawler principal, responsable de la recolección inicial de datos, y el Crawler Dinámico, que actualiza el conocimiento del sistema bajo demanda. Ambos componentes son fundamentales para mantener una base de conocimiento actualizada y relevante en el dominio de la coctelería.

Crawler Principal basado en Optimización de Colonia de Hormigas

Para la recolección inicial de datos, implementamos una aproximación novedosa basada en el algoritmo de Optimización de Colonia de Hormigas (ACO) [1]. Esta elección se justifica por la necesidad de explorar eficientemente el espacio web, priorizando las fuentes más relevantes y adaptando la estrategia de exploración según los resultados obtenidos.

El algoritmo ACO implementado presenta las siguientes características:

- **Rastro de feromonas:** Se mantiene un mapa asociativo que asigna a cada URL un nivel de feromonas, representando su relevancia histórica. Los niveles de feromonas se inicializan con un valor base de 0.1.
- **Heurística de relevancia:** Cada URL candidata se evalúa mediante una función heurística que considera:
 - Presencia de palabras clave relacionadas con coctelería tanto en la URL como en el texto ancla.
 - Pertenencia a dominios de confianza previamente identificados (con ponderaciones específicas).
 - Ausencia de patrones incluidos en lista negra (publicidad, contenido irrelevante, etc.).
- **Selección probabilística:** La probabilidad de seleccionar una URL para exploración combina su nivel de feromonas (experiencia histórica) y su valor heurístico (potencial estimado), según la fórmula clásica de ACO: $P(url) \propto (feromonas)^\alpha \cdot (heuristica)^\beta$, donde α y β controlan el balance entre experiencia histórica e información heurística.
- **Diversificación:** Se implementan dos estrategias complementarias para garantizar diversidad:
 - Limitación del número máximo de URLs por dominio, evitando concentración excesiva.

- Combinación de selección determinista (top-n basado en probabilidad) y selección estocástica (rueda de ruleta), favoreciendo la exploración de nuevas áreas.
- **Actualización de feromonas:** Se realiza mediante dos mecanismos:
 - *Depósito:* Después de procesar una página, se añaden feromonas proporcionales a la calidad del contenido encontrado: $\tau_{ij} \leftarrow \tau_{ij} + \Delta\tau_{ij}$, donde $\Delta\tau_{ij} = \text{calidad} \times \text{factor_depósito}$.
 - *Evaporación:* Periódicamente se aplica la reducción de feromonas según el factor de evaporación: $\tau_{ij} \leftarrow (1 - \rho) \times \tau_{ij}$, donde ρ es el factor de evaporación (típicamente 0.1).
- **Evaluación de relevancia de contenido:** Para medir la calidad de las páginas extraídas, se aplican múltiples criterios:
 - Densidad de palabras clave en el título y contenido.
 - Diversidad de palabras clave (número de términos únicos relacionados con coctelería).
 - Presencia de patrones típicos de recetas (ingredientes, preparación, instrucciones).
 - Bonificación para términos premium (nombres de licores, técnicas específicas).

Justificación técnica: La elección de ACO como metaheurística para el crawler responde a varias consideraciones:

- Proporciona un equilibrio natural entre explotación (seguir rutas prometedoras) y exploración (descubrir nuevas áreas).
- Se adapta dinámicamente a los cambios en el entorno web y en la relevancia de las fuentes.
- Permite incorporar conocimiento específico del dominio a través de la función heurística.
- Muestra comportamiento emergente que conduce a la identificación de fuentes valiosas no evidentes inicialmente.

Crawler Dinámico para Verificación y Actualización Como complemento al crawler principal, se implementó un crawler dinámico que actúa bajo demanda cuando el sistema detecta que una respuesta es potencialmente incorrecta o incompleta. Este crawler se caracteriza por:

- **Verificación de respuestas:** Analiza la calidad de las respuestas generadas por el sistema mediante un LLM especializado (Mistral Medium), determinando si son:
 - *Adecuadas:* Correctas y completas.
 - *Inadecuadas por incorrección:* Contienen errores factuales.
 - *Inadecuadas por incompletitud:* Correctas pero incompletas.
 - *Inadecuadas por ambos motivos:* Incorrectas e incompletas.
- **Búsqueda web inteligente:** Cuando se detectan respuestas inadecuadas, realiza búsquedas web específicas utilizando:

- Detección automática del idioma de la consulta (español/inglés).
- Reformulación inteligente de la consulta para optimizar resultados.
- Aplicación de modificadores específicos según el tipo de consulta (histórica, receta, etc.).
- Integración con motores de búsqueda externos (DuckDuckGo) para minimizar restricciones de uso.
- **Extracción de información relevante:** Procesa los resultados de búsqueda para extraer solo la información más pertinente:
 - Aplicación de prompts específicos según la naturaleza de la consulta y el idioma detectado.
 - Reducción del contexto a puntos clave para optimizar la precisión del LLM.
 - Establecimiento de parámetros restrictivos de generación (temperatura baja, top-p limitado) para obtener respuestas factuales.
- **Generación de respuestas mejoradas:** Combina la información original con la nueva para generar respuestas optimizadas:
 - Diferentes estrategias según si la respuesta original estaba vacía o era inadecuada.
 - Prompts adaptados al tipo de consulta (histórica, receta, informativa general).
 - Control estricto de extensión para mantener respuestas concisas y directas.
 - Procesamiento bilingüe, generando respuestas en el mismo idioma de la consulta.

2.3. Procesamiento del Lenguaje Natural

El procesamiento del lenguaje natural (NLP) constituye un componente central de nuestro sistema, implementado en varios niveles para permitir la comprensión, transformación y generación de información relacionada con el dominio de la coctelería. Las diferentes técnicas de NLP se han integrado en los siguientes componentes del sistema:

Vectorización Semántica Para la representación vectorial de documentos y consultas se implementó un enfoque sofisticado basado en embeddings contextuales usando modelos transformers de última generación. Como núcleo de esta implementación, se seleccionó el modelo `all-mpnet-base-v2` de Sentence-Transformers por su óptimo equilibrio entre rendimiento semántico y eficiencia computacional. Este modelo, fundamentado en la arquitectura MPNet, incorpora capacidades de procesamiento contextual bidireccional que permiten capturar relaciones semánticas sutiles entre términos y conceptos del dominio de la coctelería. Para manejar documentos extensos de manera eficiente, implementamos una estrategia de chunking inteligente que divide los textos en fragmentos más pequeños con un solapamiento controlado, técnica que preserva la coherencia semántica entre fragmentos adyacentes y evita la pérdida de información

crucial en los límites. Los vectores resultantes son sometidos a un proceso de normalización vectorial para facilitar comparaciones de similitud coseno computacionalmente eficientes, aspecto fundamental para la recuperación precisa de documentos relevantes durante la búsqueda. Para optimizar el almacenamiento y consulta de estos vectores, integramos la biblioteca FAISS que proporciona estructuras de indexación avanzadas, permitiendo realizar búsquedas aproximadas por similitud con complejidad computacional sub-lineal y minimizando los requerimientos de memoria incluso al escalar a grandes volúmenes de documentos.

Procesamiento de Consultas El sistema implementa un pipeline sofisticado de procesamiento para las consultas de usuario en lenguaje natural, integrando diversas técnicas complementarias. En la primera fase, se aplica un mecanismo de detección automática de idioma que analiza patrones léxicos y la presencia de términos específicos para determinar si la consulta está formulada en español o inglés, adaptando así todo el procesamiento posterior a las particularidades lingüísticas del idioma identificado. Para optimizar la efectividad de la recuperación de información, implementamos un sistema de expansión de consultas que enriquece la entrada original con términos semánticamente relacionados según el contexto detectado, ya sea histórico, de recetas, ingredientes u otras categorías contextuales relevantes en el dominio de coctelería. Las consultas procesadas son posteriormente transformadas en representaciones vectoriales mediante el mismo modelo de embedding utilizado para los documentos, garantizando así la compatibilidad semántica en el espacio vectorial compartido y facilitando comparaciones precisas de similitud. Complementando estas técnicas, implementamos un análisis de intención que identifica el propósito subyacente de cada consulta—ya sea búsqueda de recetas, indagación histórica, solicitud de sustitución de ingredientes u otros—permitiendo dirigirla al flujo de procesamiento más adecuado dentro del sistema multiagente.

Generación de Respuestas La generación de respuestas textuales constituye una fase crítica en nuestro sistema, implementada mediante la integración de técnicas avanzadas de procesamiento del lenguaje natural. Desarrollamos una metodología de prompting especializado, con plantillas cuidadosamente diseñadas y optimizadas para cada categoría específica de consulta y situación contextual, incluyendo variantes diferenciadas para los procesos de verificación, búsqueda, extracción de información y generación final de respuestas. Un aspecto fundamental de nuestra implementación es el meticuloso control de los parámetros de generación, donde valores como temperatura, top-p y longitud máxima son ajustados dinámicamente según el contexto específico de uso, priorizando la exactitud factual en consultas de carácter informativo mientras permitimos cierto grado de creatividad controlada en sugerencias y recomendaciones. Para garantizar la calidad de las respuestas, implementamos un riguroso proceso de verificación posterior a la generación que evalúa tanto la corrección técnica como la completitud de la información proporcionada, con la capacidad de activar automáticamente el crawler dinámico cuando se detectan deficiencias informativas.

Finalmente, aplicamos técnicas avanzadas de formateo contextual que adaptan automáticamente cada respuesta al idioma original de la consulta y estructuran la información según la naturaleza específica del contenido, ya sea histórico, instructivo (recetas), comparativo u otro formato adecuado a las necesidades del usuario.

Extracción de Conocimiento para la Ontología El sistema incorpora un conjunto integrado de técnicas de extracción de información para poblar sistemáticamente la ontología de coctelería. Implementamos mecanismos avanzados de reconocimiento de entidades que identifican automáticamente elementos relevantes del dominio—como cócteles, ingredientes, herramientas y técnicas—mediante la combinación de patrones lingüísticos predefinidos y modelos específicos de Named Entity Recognition (NER). Complementando este proceso, desarrollamos algoritmos de extracción de relaciones semánticas que detectan conexiones significativas entre las entidades identificadas, aplicando análisis sintáctico de dependencias y evaluación contextual para determinar la naturaleza precisa de cada relación. La información extraída se somete a un proceso de transformación estructurada que convierte los datos en tripletas RDF (sujeto-predicado-objeto), incorporando procesos de normalización terminológica y verificación de coherencia lógica antes de su integración definitiva en la ontología formal. Para facilitar la interacción con este conocimiento estructurado, implementamos un sistema de procesamiento de consultas que traduce preguntas formuladas en lenguaje natural a consultas ejecutables en SPARQL mediante un sistema de plantillas adaptativas, permitiendo así interrogar la ontología de manera flexible y precisa.

La integración armoniosa de estas técnicas avanzadas de procesamiento del lenguaje natural en los diferentes componentes del sistema permite comprender consultas complejas formuladas en lenguaje natural, recuperar información contextualmente relevante de múltiples fuentes heterogéneas [4], razonar eficazmente sobre el conocimiento estructurado en la ontología, y finalmente generar respuestas coherentes y contextualizadas que se adaptan tanto al perfil específico de cada consulta como a las preferencias implícitas del usuario. Esta combinación de capacidades lingüísticas computacionales constituye uno de los pilares fundamentales para el funcionamiento integral del sistema Cocktail Assistant.

2.4. Representación del Conocimiento: Enfoque Híbrido

En el diseño de Cocktail Assistant, la representación del conocimiento juega un papel fundamental para facilitar la recuperación precisa de información y el razonamiento sobre el dominio. Adoptamos deliberadamente un enfoque híbrido que combina dos paradigmas complementarios: representación semántica estructurada (ontologías) y representación vectorial distribuida (embeddings). Esta decisión responde a la necesidad de abordar diferentes tipos de consultas y razonamiento.

Ontología vs. Embeddings Vectoriales: Análisis Comparativo Ambos enfoques presentan fortalezas y limitaciones específicas que influyen directamente en su idoneidad para diferentes aspectos del sistema:

Cuadro 1. Comparación entre representaciones del conocimiento

Aspecto	Ontología (RDF/OWL)	Embeddings Vectoriales
Estructura	Explícita, formal y lógica	Implícita y distribuida
Razonamiento	Inferencia lógica, transitividad	Similaridad semántica, analogías
Consultas	Precisas, basadas en reglas (SPARQL)	Aproximadas, basadas en similitud
Actualización	Requiere consistencia lógica	Flexible, sin restricciones formales
Escalabilidad	Limitada por complejidad de inferencia	Alta, paralelizable

Justificación del Enfoque Híbrido La decisión de implementar ambas representaciones en paralelo se fundamenta en diversas consideraciones metodológicas y prácticas. Analizando la naturaleza del dominio, observamos que la coctelería contiene una dualidad inherente: por un lado, conocimiento estructurado y categórico (ingredientes, proporciones, tipos) que se representa óptimamente mediante ontologías; por otro lado, conocimiento implícito y contextual (descripciones, historia, experiencia sensorial) que requiere la flexibilidad de los embeddings vectoriales. Esta dualidad se refleja también en la diversidad de consultas que los usuarios formulan, desde preguntas factuales precisas (“¿Qué ingredientes lleva un Mojito?”) hasta consultas exploratorias y abiertas (“Sugiere cócteles refrescantes para el verano”).

El enfoque híbrido permite además implementar mecanismos de razonamiento complementario, donde ciertas inferencias requieren el razonamiento formal que ofrecen las ontologías (como la sustitución de ingredientes basada en categorías taxonómicas), mientras que otras se benefician de la similitud semántica captada por los embeddings (como recomendaciones basadas en preferencias previas o similitudes no explícitamente codificadas). Finalmente, este enfoque dual proporciona robustez ante información incompleta: cuando la ontología carece de información específica sobre algún concepto, los embeddings pueden proporcionar respuestas aproximadas basadas en similitud contextual, y viceversa, creando un sistema resiliente ante las inevitables lagunas de conocimiento.

Implementación de la Representación Dual La arquitectura del sistema integra ambas representaciones de la siguiente manera:

- **Ontología estructurada:** Implementada en RDF/OWL, con las siguientes características:
 - Jerarquía clara de clases (Cócteles, Ingredientes, Métodos, etc.)
 - Relaciones explícitas y tipadas (contiene, sirveEn, preparadoCon)
 - Propiedades de datos (graduación alcohólica, año de creación)

- Restricciones lógicas (un Daiquiri requiere ron como ingrediente principal)
- Consulta mediante SPARQL para preguntas precisas con respuestas definidas
- **Representación vectorial:** Implementada con embeddings de transformers:
 - Captura de relaciones semánticas implícitas entre conceptos
 - Representación de conocimiento contextual y matizado
 - Capacidad para manejar consultas aproximadas o mal formuladas
 - Identificación de similitudes no evidentes en la estructura formal
 - Indexación mediante FAISS para búsqueda eficiente por similitud

Integración y Selección Dinámica El agente de estrategia del sistema es responsable de determinar qué representación del conocimiento es más adecuada para cada consulta:

- **Análisis de consulta:** Clasifica las preguntas según parámetros como precisión requerida, estructura, disponibilidad de datos y objetivos del usuario.
- **Selección ponderada:** En algunos casos, combina resultados de ambas fuentes con diferentes pesos según la confianza en cada representación.
- **Reconciliación:** Resuelve posibles inconsistencias entre los resultados obtenidos de ambas representaciones.
- **Retroalimentación:** Actualiza la estrategia de selección basándose en la efectividad de respuestas anteriores.

Esta aproximación híbrida ha demostrado ser particularmente efectiva para manejar la diversidad de conocimiento en el dominio de la coctelería, permitiendo tanto respuestas precisas y confiables a preguntas específicas como exploraciones creativas y recomendaciones personalizadas, extendiendo significativamente las capacidades de recuperación de información y asistencia del sistema.

3. Autocrítica y Limitaciones

El análisis crítico de nuestra implementación revela tanto fortalezas significativas como áreas que requieren mayor desarrollo. Esta reflexión autocrítica es esencial para el proceso de mejora continua del sistema y para orientar investigaciones futuras.

3.1. Logros Significativos

El proyecto Cocktail Assistant ha alcanzado avances notables que merecen destacarse:

- **Integración tecnológica:** Se ha conseguido una síntesis efectiva de técnicas avanzadas de inteligencia artificial (RAG, modelos de lenguaje, ontologías) con algoritmos metaheurísticos (ACO, algoritmos genéticos) en una arquitectura multiagente cohesiva y funcional.

- **Precisión en el dominio específico:** El sistema ha demostrado una comprensión profunda del dominio de la coctelería, evidenciada en respuestas técnicamente precisas sobre recetas, ingredientes, sustituciones y clasificaciones.
- **Capacidad inferencial:** Destacable habilidad para realizar razonamientos sofisticados como sustitución de ingredientes basada en propiedades organolépticas, combinaciones compatibles y adaptaciones según restricciones.
- **Verificación y actualización:** El mecanismo de verificación de respuestas y actualización dinámica del conocimiento representa un avance significativo respecto a sistemas tradicionales, reduciendo la generación de información incorrecta.
- **Adaptabilidad:** La arquitectura ha demostrado flexibilidad para incorporar nuevas fuentes de conocimiento y adaptarse a distintos tipos de consulta sin requerir reentrenamiento completo.

3.2. Limitaciones Técnicas

Sin embargo, hemos identificado varias limitaciones técnicas que ameritan reconocimiento y consideración:

- **Integración de representaciones del conocimiento:** A pesar del enfoque híbrido, el sistema presenta dificultades con consultas complejas que requieren combinar simultáneamente información de embeddings y ontologías:
 - Las representaciones operan de forma independiente, sin un mecanismo real de integración en tiempo de consulta
 - La selección entre uno u otro método se realiza al inicio del procesamiento, sin poder cambiar dinámicamente
 - Consultas que requieren tanto inferencia lógica como similaridad semántica pueden producir respuestas incompletas
- **Latencia del crawler dinámico:** Las búsquedas en tiempo real del crawler dinámico pueden introducir latencias significativas, afectando la experiencia de usuario en consultas que requieren verificación externa.
- **Ausencia de contexto conversacional:** El sistema actual está limitado a consultas independientes:
 - No mantiene histórico de interacciones previas
 - No puede resolver referencias anafóricas entre consultas
 - No se adapta al perfil del usuario a lo largo de la conversación

3.3. Direcciones Futuras

Para abordar las limitaciones identificadas, proponemos un plan de implementación concreto que comprende las siguientes líneas de desarrollo:

- **Integración unificada de representaciones:** Para resolver la problemática de integración entre embeddings y ontologías, proponemos:

- *Implementación*: Desarrollo de una capa de abstracción mediante un *Knowledge Integration Service* que:
 - Traduzca consultas a un formato canónico procesable por ambos sistemas
 - Ejecute consultas en paralelo en ambas representaciones con ponderaciones dinámicas
 - Aplique funciones de reconciliación para resolver inconsistencias entre resultados
- *Arquitectura técnica*: Creación de un pipeline de procesamiento con tres fases:
 - *Descomposición*: Análisis de la consulta para identificar componentes que requieren razonamiento lógico vs. similaridad semántica
 - *Ejecución paralela*: Distribución de sub-consultas a motores de ontología y vectoriales
 - *Fusión adaptativa*: Combinación de resultados mediante algoritmos de rank-fusion con aprendizaje por refuerzo
- *Validación*: Implementación de un sistema de verificación cruzada que:
 - Compare resultados de ambas representaciones
 - Detecte y resuelva contradicciones mediante reglas de preferencia configurables
 - Actualice parámetros de fusión basados en retroalimentación de precisión
- **Sistema de gestión conversacional**: Para superar la actual limitación de consultas independientes:
 - *Componentes clave*:
 - *Gestor de estado de diálogo*: Almacena y actualiza el contexto conversacional
 - *Módulo de resolución de referencias*: Procesa referencias anafóricas y elipsis
 - *Procesador de intenciones*: Identifica la evolución de intenciones a lo largo del diálogo
 - *Procesamiento contextual*:
 - Transformación de consultas dependientes del contexto a consultas autocontenidas
 - Mantenimiento de entidades mencionadas con sus atributos relevantes
 - Detección y manejo de cambios temáticos en la conversación
 - *Personalización progresiva*:
 - Construcción de perfiles de usuario basados en preferencias detectadas
 - Adaptación del nivel técnico de respuestas según interacción previa
 - Sistema de clarificaciones cuando las consultas son ambiguas en el contexto
 - *Implementación*:
 - Extensión del modelo de mensaje para incluir metadatos de sesión y contexto

- Integración con el broker de mensajes existente
- Base de datos temporal para almacenar estados conversacionales con TTL configurable

4. Conclusiones

El sistema Cocktail Assistant demuestra la viabilidad y eficacia de una arquitectura multiagente para recuperación de información especializada en coctelería [5]. Nuestra investigación ha resultado en varias contribuciones significativas al campo. Primero, hemos implementado un modelo RAG especializado [2] que logra un equilibrio óptimo entre la recuperación contextual de información relevante y la generación controlada de respuestas coherentes. Segundo, adaptamos el algoritmo de Optimización de Colonia de Hormigas (ACO) [1] para permitir una exploración web eficiente que optimiza la búsqueda de fuentes relevantes mientras minimiza recursos computacionales. Tercero, desarrollamos un enfoque híbrido de representación del conocimiento que combina la flexibilidad de los embeddings vectoriales con la precisión estructural de las ontologías, permitiendo respuestas más precisas y contextualizadas. Finalmente, implementamos un sofisticado sistema de verificación que reduce significativamente la generación de información incorrecta, aumentando la confiabilidad del sistema.

La arquitectura desarrollada trasciende el dominio específico de la coctelería, siendo transferible a otros campos especializados donde coexisten conocimiento estructurado y no estructurado. A pesar de las limitaciones identificadas en la integración de representaciones y el manejo contextual de diálogos, este enfoque híbrido sienta las bases para el desarrollo de asistentes inteligentes con verdadero valor práctico en dominios de conocimiento especializados.

Referencias

1. Dorigo, M., Stützle, T.: Ant colony optimization: Basic concepts and applications. *Revista de Inteligencia Artificial* (2004)
2. Lewis, P., Perez, E., otros: Retrieval-augmented generation for knowledge-intensive tasks. *Neural Information Processing Systems* (2020)
3. Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.t., Rocktäschel, T., Riedel, S., Kiela, D.: Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in Neural Information Processing Systems* 33, 9459–9474 (2020)
4. Manning, C.D., Raghavan, P.: *Introduction to Information Retrieval*. Cambridge University Press (2008)
5. Wooldridge, M.: *Introduction to Multiagent Systems*. Wiley (2009)